Real-time applications on the Internet

S Rudkin, A Grace and M W Whybray

This paper provides a general introduction to real-time applications over networks operating the Internet protocol. A particular issue facing such applications is the difficulty in consistently achieving the required quality of service. This paper looks at the role of the network, the operating system and the application environment in delivering the required quality of service. The main focus is an outline description of an application architecture designed to support a wide range of real-time applications, with particular reference to audiovisual services.

1. Introduction

1.1 What are real-time applications?

A real-time application must include some time critical element. This could be a discrete message such as a mouse movement controlling a shared whiteboard. Or it could be an isochronous media stream, i.e. a continuous stream of bits with strict time dependencies between those bits.

When describing real-time applications it is useful to divide them into conversational applications and streamed applications. Conversational applications are primarily interactive and will usually have humans present at each end. Typical applications include Internet telephony [1], audio or video or data conferencing, application sharing, text-based chat, networked games, shared virtual worlds or distributed simulations.

Streamed applications are essentially one way flows of information. Typical examples would include information services such as stock prices, or traffic information, and broadcast or on-demand video and audio services.

1.2 End-to-end quality of service

Real-time Internet applications have quality of service (QoS) requirements that must be considered on an end-to-end basis [2]. In taking an end-to-end perspective, end-system and network capabilities are equally important in delivering the QoS support required at the application layer, as shown in Fig 1.

For real-time media, application designers are primarily concerned with temporal properties such as delay, jitter, bandwidth and synchronisation, and reliability properties such as error-free delivery, ordered delivery, fairness. Desired values for these QoS parameters are determined by the limits of human perception.

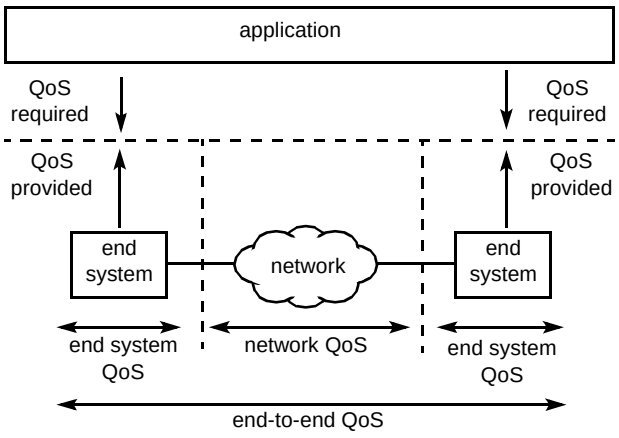

Fig 1 End-to-end quality of service.

Audio quality is highly sensitive to jitter, and the watchability of video is sensitive to available bandwidth [3]. For conversational services acceptable delays are in the order of no more than a few hundred milliseconds. For example, once round trip speech delays exceed 400 ms, conversation can become disjointed [4].

For games or distributed interactive simulations, even more stringent delay targets in the order of 100 ms are required on certain critical interactions [5]. For lip synchronisation, audio and video streams need to be synchronised to within 80 — 100 ms if the skew is to be imperceptible [6].

Discrete interactions tend to have more stringent reliability requirements than isochronous interactions. In general, a discrete interaction is processed in a way that depends on the computer deciding how to present its meaning to the user. Since a discrete interaction may consist only of a single packet, the loss of that packet can have a major impact on the application's behaviour and the user's perception.

In contrast, for an isochronous stream the computer makes no such decision; packets are effectively passed automatically through to the presentation device. Interpretation of the delivered information is left to human perception; and as humans are far more tolerant than computers, lost packets are likely to be perceived merely as a temporary reduction in quality of the signal. Consequently techniques supporting reliability play a more critical part in the implementation of discrete interactions than of isochronous interactions. Reliability techniques for discrete interactions are discussed in section 4.3 .

Nevertheless packet loss is still a significant problem for isochronous interactions. For example, since a typical packet size is generally above the threshold for audible loss (around 20 ms), the loss of a single packet will be audible [7]. Furthermore uncorrected errors in a video stream would cause gross degradation of the picture (such as a random array of nicely coloured blocks!). These are so visible and annoying that some error concealment scheme is usually considered mandatory, the simplest of which is to freeze the last good picture until the video is recovered in some way.

The remainder of the paper looks at the role of the network (section 2), the operating system (section 3) and the application environment (section 4) in delivering the quality of service required by the application.

2. Network architecture

The integrated services architecture (ISA) is a suite of standards under development within the IETF [8, 9]. Its aim is to offer control over the quality of service (in terms of delay, jitter and bandwidth) provided by transport over an IP network.

Traditionally the Internet has provided a best-effort service offering no timeliness or bandwidth guarantees. Typically routers support a best-effort service using FIFO queuing, with the consequence that all streams may be subject to packet loss during congestion. ISA addresses this by allowing certain flows to take priority over others. This is based on the recognition that traffic is broadly a mixture of elastic traffic (which can tolerate delays and discards) and inelastic traffic (which cannot). ISA defines three classes of service:

- guaranteed service permits applications to obtain both bandwidth and delay guarantees,
- controlled load service makes delays minimal by prioritising controlled load streams over best-effort streams,
- best-effort service is just the traditional IP service available today on the Internet.

Classes of service are assigned to streams of traffic called flows, through a process known as classification. Since IPv4 does not support flow identifiers, flows are distinguished by the source and destination addresses, port numbers and protocol type.

The resource reservation set-up protocol RSVP is used to classify flows and to reserve network resources for that flow [10]. Individual network elements (subnets and IP routers), along the path followed by a flow, support mechanisms to control the quality of service delivered to the flow[1].

Guaranteed service requires that the network supports both admission control and an appropriate scheduling algorithm such as weighted fair queuing. Admission control ensures that that there are sufficient available resources before accepting a reservation. Weighted fair queuing is one way for routers to schedule packets in accordance with the queuing policy that was installed when the reservation was placed.

One possible implementation of controlled-load service is to provide a queuing mechanism with two priority levels — a high priority one for controlled-load and a lower priority one for best-effort service. An admission control algorithm is used to limit the amount of traffic placed into the high-priority queue.

RSVP is designed to operate with IP multicast [11]. A multicast flow can be received by many receivers. Receivers choose to join or leave a multicast group by sending Internet group management protocol (IGMP) [11] messages to their local router. Once a receiver has joined a multicast group they will receive any information sent to the group's multicast address. IP multicast enables the network to build a routeing tree for efficiently distributing information to all receivers within the associated multicast group. Because information sent to all receivers only traverses each branch of the tree just once it is extremely efficient and highly suited to the distribution of flows of bandwidth-hungry media such as audio and video.

[1] RSVP and associated mechanisms for QoS control need not be operating across the length of the flow's path. A reservation is still beneficial in such cases if RSVP and supporting mechanisms for QoS control are operating in the congested region of the flow's path.

Like IP multicast, RSVP is also receiver based, i.e. it is receivers not senders that make reservations. This allows different receivers within a multicast session to have in place different reservations, Consequently, some receivers, for example, might reserve resources for the audio stream only, others might reserve resources for black and white video, and yet others might reserve resources for full colour.

The real-time transport protocol (RTP) is the transport protocol for real-time traffic [12], satisfying a number of requirements:

- it allows separate media to be handled as individual streams — if required these can be multiplexed at the IP layer,
- it includes time-stamps facilitating synchronisation — the time stamps are media specific, i.e. they are in units that are appropriate to the media stream,
- it provides a basic framing service allowing the application to define a frame as the unit of synchronisation in accordance with the principle of application layer framing [13] — this says that it is the application that is best placed to determine how to divide the data stream into frames, because only the application knows how the data will have to be processed on receipt,
- it labels streams with a synchronisation source identifier so that different streams from a single machine can be distinguished — as all packets from a synchronisation source form part of the same timing and sequence number space, a receiver groups packets by synchronisation source for playback; RTP also allows streams from several sources to be mixed in a gateway to produce a single stream, each packet in the resulting stream carrying source IDs of all contributing streams.

Associated with each RTP flow there are control packets, sent using the real-time control protocol (RTCP) [12]. RTCP 'sender reports' map the system clock at the sender to the RTP media time-stamps. RTCP 'source description' packets provide a canonical name in textual form, allowing the sender in a conference to be identified from the synchronisation source identifier. Where the sender is transmitting multiple streams, these packets serve to associate the different sources with the canonical name. RTCP 'receiver reports' are used in a multicast conference to indicate the current conference membership and to provide information about reception quality.

From an application programmer's perspective the most important part of the integrated services architecture is the interface it presents to the programmer — called the network API in Fig 2. Currently this is a collection of APIs. For example the Windows Sockets API Winsock 1.1 [14] supports TCP/UDP/IGMP [2]. The developing Winsock 2.0 API will support RSVP [15, 16]. Precept Software Inc has developed an API and Software Development Kit for RTP and RTCP.

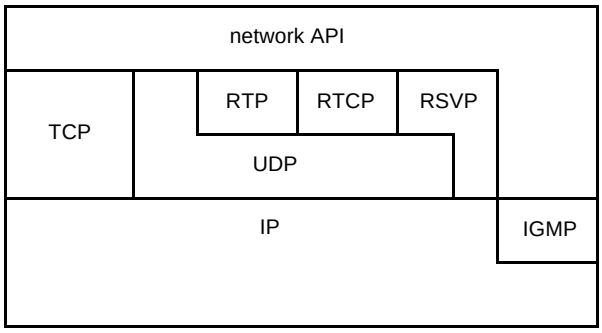

Fig 2 Network architecture.

3. Operating system architecture

Unlike the network architecture, it is not possible to point to a major standards initiative which is addressing QoS support in operating systems. Consequently this section will restrict itself to a brief discussion of operating system support for real-time applications.

3.1 Hardware or software processing?

Inside the end-system, real-time media can be processed by either hardware or software. For example, an MPEG video stream can be decoded using a dedicated hardware decoder, where the operating system is responsible for controlling the data stream. Alternatively, the MPEG video stream could be decoded using a software decoder, in which case the operating system is responsible for both controlling and processing the data stream.

In terms of performance, the speed at which a hardware decoder can operate is historically higher than that of a software decoder. A dedicated hardware decoder can also deliver predictable performance, whereas a software decoder running on a non-dedicated CPU may have to compete with other processes, and as a result be unable to process media within guaranteed temporal bounds.

Concerns over the end-system component of the end-to-end QoS problem can be greatly relaxed if real-time data never touches the operating system or the CPU, but passes straight to, from or between specialised hardware devices. However, despite the performance penalty, software processing of real-time data through the operating system brings about a number of opportunities:

- video presentation capabilities can be added to a workstation or PC at almost zero cost, as no additional hardware is required,

[2] IGMP is supported in extensions to Winsock 1.1 and implemented in Windows 3.11 and later.

- multiple media streams can be handled by the application — video from multiple sources can be displayed simultaneously whereas a typical hardware decoder can only support a single video window — multiple audio sources can be switched or mixed by the application to a speaker,

- advanced media processing functions, such as speech recognition or image categorisation, can be integrated into real-time applications.

3.2 Requirements on the operating systems

Deterministic operating system behaviour

For real-time media processing in software, the challenge at the operating system level is not simply the handling of high data rates or processing loads. Non-real time graphics rendering or transaction processing applications may have data throughput requirements that are equal to or even above those imposed upon the operating system by conversational audio and video. The difference is with the timing requirements of real-time applications and with the need for deterministic behaviour from the operating system.

Increased processing power

The trend for CPU processor performance is for computational power to increase and cost to fall. However, although processors may run faster, having additional raw CPU MIPS performance available does not in itself bring about any increase in the predictable performance of operating systems. Additionally, continuous media induces stress into areas of the CPU architecture where performance has not increased in line with MIPS power, like interrupt latencies, context switching, and data movement [17]. In order to reduce packetisation delay, small packet sizes are typically used for audio. A 20-ms speech packet sampled at 8 kHz and coded as μ-law PCM is 160 bytes long. Yet while small packet sizes will reduce end-to-end delay, they generate a large interrupt load for network device drivers and frequent context switches for applications.

Scheduling characteristics should match media characteristics

For application QoS requirements to be satisfied by the end-system, the operating system must be able to schedule a number of tasks or processes, each with its own characteristic behaviour. In a simple audio and video conferencing application, the scheduling requirements for incoming or outgoing media streams can be characterised as follows:

- audio capture and encoding is both periodic and predictable, with activity occurring at a known sample rate, and likely to have consistent processing requirements from sample to sample,

- audio decoding and presentation is also both periodic and predictable (providing the source is still transmitting),

- video capture and encoding has either a fixed or a variable period depending upon the coding scheme used, e.g. MPEG operates at a fixed frame rate, whereas H.261/3 has a dynamically varying frame rate to accomodate the wide variation in data generated as a function of motion and complexity at a particular point in a sequence — the processing requirements vary from frame to frame as a function of image content and coding scheme, and will tend to be unpredictable,

- video decoding and presentation has either a fixed or variable period depending upon the coding scheme used and incoming frame rate — the image decoding process again places an unpredictable load on the operating system due to the impact of image content.

The requirement for synchronisation between multiple media streams introduces further scheduling complexity for the decoder, with different scheduling characteristics to both audio and video decoding.

3.3 Real-time scheduling schemes

Support for real-time QoS can be provided by the operating system through the use of different scheduling schemes:

- fixed priority, where each task has a static priority assigned to it and tasks are scheduled according to their priority,

- dynamic priority, where the priority of a task varies during its lifetime — a popular dynamic priority scheduling algorithm is based upon earliest-deadline first scheduling [18], where the task with the earliest deadline is scheduled.

As fixed-priority scheduling enables real-time tasks to be distinguished from non-real-time activities, decoding and playing incoming audio packets, for example, can be given a higher priority than responding to keyboard input.

Deadline scheduling is very attractive to real-time continuous media applications, as there can be natural deadlines associated with decoding and displaying frames. For example, with MPEG at 5 frames/s, a deadline for presenting a video frame occurs every 200 ms. As a deadline approaches, so the priority of performing an activity increases. If a deadline is missed, the value of

completing the activity is reduced possibly to the point where under some circumstances it is not worth completing it.

Existing operating systems and hardware architectures provide limited support for multimedia applications. Desktop operating systems such as MacOS, Windows3.1 provide little or no QoS support; Windows95 and Windows NT provide support for four scheduling priority classes. However, general-purpose real-time multimedia operating systems are still the subject of research [19, 20].

4. Applications architecture

Timeliness and reliability requirements are very often in tension. For example, re-transmissions result in increased end-to-end delay and increased jitter. For effective QoS control, systems should deliver the optimum balance between these conflicting requirements.

The suggested solution is to adopt a loosely coupled approach as promoted by van Jacobson and exemplified by applications such as vic (videoconferencing) [21], vat (audio conferencing) [22] and wb (shared whiteboard) [23]. This approach is more scalable than tightly coupled approaches and also is not subject to central points of failure[3]. It allows for a flexible architecture supporting a fast but unreliable service which can be extended to include additional reliability where necessary. A typical system consists of a distributed collection of hosts communicating via IP multicast. Each host is structured according to the applications architecture of Fig 3.

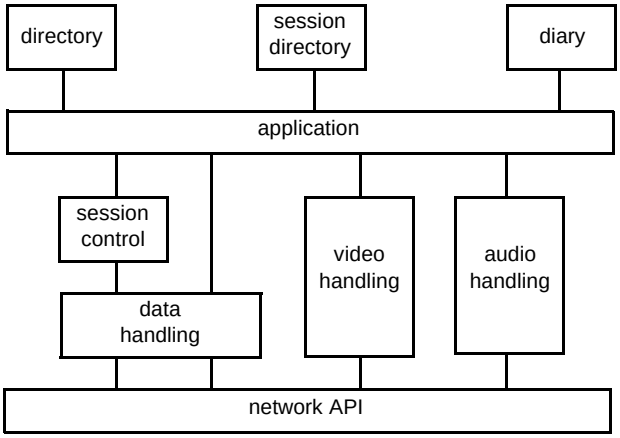

Fig 3 Applications architecture.

[3] A useful scenario for understanding loose versus tightly bound models is to consider an open meeting where there are a small number of active participants but potentially large numbers of observers. Tight coupling may be appropriate between the key players where delivery and consistency guarantees are important, but where the number of key players is low enough that performance is not at stake. However, to avoid compromising performance for the large number of observers, they should be loosely coupled.

The architecture shows the application interacting with four key objects and potentially making use of support tools such as a personal directory, a session directory or a diary. The four key objects are:

- the session-control object establishes the necessary channels of communication between the various media handling objects and their peers in other locations,
- the data-handling object manages discrete interactions between peer applications according to the QoS requirements specified by the application object — it is also used by the session-control object,
- the audio-handling object manages transfer of audio streams between the audio device and the network,
- the video-handling object manages the transfer of video streams between the grabber/display and the network.

'Application' is used in Fig 3 in its most general sense; it may be any of the applications mentioned in section 1. It should be noted, though, that the generic media-handling and session-control capabilities are separated from the application and encapsulated in the four key objects. Under this architecture vic, vat and wb would each be split into an application object (which would include the user interface), a session-control object and the appropriate media-handling object.

The four key objects are discussed further in subsequent sections. There then follows a brief discussion of intra-media synchronisation and support tools (such as a personal directory, a session directory and a diary).

4.1 Session control

The session-control object establishes the necessary channels of communication between the various media-handling objects. Its main responsibilities are:

- call control — establishing a connection between peer session control objects, e.g. this may be point-to-point call set-up, or may be a join to a multicast group,
- stream control — establishing media connections,
- QoS control — making appropriate resource reservations or adapting to available resources based on QoS requirements governing the application associations,
- media control — inclusion or exclusion of media-handling objects and their co-ordination,
- distributing membership information of a session,
- invoking security and accounting procedures.

The discrete interactions needed to carry out the above responsibilities will have their own QoS requirements, hence these interactions require the support of the data handling object described in section 4.2 .

Separation of call set-up from stream control has the benefit that call set-up between two parties can be initiated by a third party or by a different application. This facilitates such applications as web control of calls (browser requests *A* and *B* be connected), conferencing (*A* requests that *B* be connected to conference *C*), automatic call distribution (request for connection to *A* is passed to an application that dynamically translates the request into a request for connection to *B*).

4.1.1 Call control

Call Control includes:

- session creation — creation of a session may or may not be combined with other aspects of call control,
- session advertisement — a session may be advertised by announcement where the session details are effectively published, or during invitation by directly passing session details to the invitee(s),
- address resolution — at some point in the call set-up the session identity must be resolved to an IP address and port,
- ability to invite other parties into a session and for those parties to accept or reject the invitation,
- ability for parties to join or leave a session,
- session tear-down.

Successful call set-up results in an end-to-end connection between the peer session-control objects.

Whether call set-up is by invitation or by joining, it must be established that the peer applications are compatible and capabilities such as compression algorithm and resilience settings must be negotiated. Sessions may include any number of parties and those parties may be users or devices such as a streaming server/recorder. Multi-party sessions can be supported using a multi-point controller such as conference bridge or using network multicast (e.g. IP multicast).

To illustrate the different ways in which the different aspects of call set-up can be combined, here are some examples:

- an H.323 Internet telephone call is an example of a session created by the caller and advertised simultaneously with an invitation to the called party to come into the session — the address resolution may take place in advance (for example, because the IP address and port of the called party are known) or it may be carried out at the time using some kind of user location service,
- a multicast session created using a tool such as SDR [24] is typically advertised in advance of the event, the multicast address of the session being allocated at the time of creation and included together with the media description(s) in the advertisement — users participate in the event by the joining the specified multicast address,
- a PSTN chat service is created and the address (or telephone number) is allocated and advertised in advance of users calling — users join by dialling in and are connected together via a conference bridge.

4.1.2 Stream control

Stream control is concerned with the establishment of media connections between participating endpoints. Negotiation of capabilities such as compression algorithms may form part of stream control.

4.1.3 QoS control

Quality control uses application-specified QoS requirements to make appropriate resource reservations or to adapt to available resources. Resource reservation includes reservation of network resources using RSVP and in the future reservation of operating system resources.

The main form of adaptation is flow control. Flow control for the point-to-point case is relatively straight forward, because the receiver can directly place back-pressure on the source causing it to reduce the data rate, for example, by reducing the frame rate or stopping transmission of higher layers of resolution in the media coding.

Flow control for multicast is a very difficult problem. Consequently there is much interest in layered multicast, which distributes different layers of a media stream to different multicast groups. Using schemes such as McCannes' receiver-driven layered multicast (RLM) [25], the receiver can respond to network conditions by dynamically joining and leaving multicast groups. This has the beneficial property that flow control is effectively localised to the part of the multicast tree that is experiencing congestion. The session control object is responsible for managing the joining and leaving of different layers of a layered multicast stream. It relies on the media-handling objects to report on conditions such as packet loss. The session-control object is also responsible for co-ordinating joins and leaves across different layered streams.

4.1.4 Media control

Media control governs the behaviour of the different media-handling objects within the session. Based on the session description, the application will have the option of involving different media-handling objects in the session. Media control is responsible for starting and stopping the media handling as appropriate. Media control also governs the use of a floor control policy such as starting transmission of video when the user starts to speak, or granting permission to speak. CCCP [26] implements a loosely coupled approach to media control. Different applications and even different uses of a particular application will typically require different floor control policies. For example, Benford and Greenhalgh's collaborative research group virtual environment (CVE) [27] is able to associate multicast groups with a virtual space. As a participant moves from one room to another they might change multicast groups — moving perhaps from a classroom discussion into a corridor discussion. Within the classroom, the teacher may want a strict floor control policy dictating who can speak at any one time. Out in the corridor there may be no control. It may be that the application programmer imposed these policies when creating the shared virtual space. Alternatively it might be the prerogative of the person who booked the space to select and impose their own policy (chosen from a range of pre-defined options). The session control object should be capable of supporting different floor control policies.

4.1.5 Exchanging membership information

Exchange of membership information is required for a variety of reasons, e.g. for monitoring interest or monitoring connectivity, RTCP can be used to exchange membership information. In defining a session control mechanism that is common to all applications and all media, a common way of representing membership information within RTCP is required.

4.1.6 Security and accounting

Security and accounting is an important area. Some discussion of security is included in section 5. Crowcroft et al discuss charging [28].

4.1.7 Standards for session control

H.323 [29] has recently been developed within the ITU as the standard for audiovisual conferencing on LANs and is designed to interoperate through gateways with H.320 (ISDN audiovisual conferencing) and H.324 (PSTN audio-visual conferencing). H.323 is gaining strong support from vendors as the standard for conversational services on IP networks in general, and not just on LANs. H.323 uses a variant of Q.931 to handle transitions between call states. Successful call completion results in the establishment of an H.245 control channel. The control channel is used to do capabilities negotiation and to do bearer control, i.e. open and close logical channels. The logical channels are used to carry data, audio or video.

H.323 defines an optional gatekeeper whose job it is to do address translation, admissions control and bandwidth control. H.323 terminals talk to the gatekeeper using the registration admission and status (RAS) signalling function. This is done prior to opening any other H.323 channels.

The gatekeeper and RAS together support user location on the LAN, but no mechanism exists for supporting user location across subnets. To support PBX style functions such as hold and retrieve an API is required to make and break H.245 logical channels. Currently hold and retrieve can only be supported through the use of Q.931 facility messages, but there is no standard to guarantee consistent interpretation of these messages.

H.323 defines a multi-point controller to provide control functions for multi-party conferences. The multi-point controller optionally interacts with a multi-point processor to indicate how streams should be mixed.

Conferences may be centralised or decentralised. For centralised conferences data, audio and video are handled by the multi-point processor. For decentralised conferences audio and video is distributed using IP multicast. T.120 [30] is the default standard for handling data within H.323.

H.323 specifies TCP for carrying Q.931, H.245 control and data and specifies UDP for carrying RAS signalling, audio and video. H.323 is therefore described as tightly coupled and suffers from the consequent scalability problems. As a result work has recently started on extensions to H.323 to support a mixture of tightly and loosely coupled conferencing. The proposal revolves around the concept of a tightly coupled H.323 panel and a loosely coupled RTP/RTCP audience. Provided they support H.323 terminals, members of the audience can be invited to join the panel. This does not provide the ideal of a smooth trade-off between reliability and timeliness but it does provide a useful combination of the two ends of the spectrum.

The **session description protocol** (SDP) is being specified by the IETF for describing sessions [31]. Specifically it communicates the existence of the session and sufficient information to enable joining and participating in the session.

SDP includes:

- the type of the media (audio, video, etc),
- the transport protocol (RTP/UDP/IP, H.320, etc),

- the format of the media (H.261 video, MPEG video, etc),
- address and port for the media.

Normally SDP is used by a session-directory tool (such as SDR [24]) that announces a session by periodically multicasting an announcement packet on a well-known multicast address and port. This multicasting of announcements is performed by the session announcement protocol (SAP) [32]. A SAP packet simply comprises a SAP header and an SDP payload. SDP announcements may also be distributed by e-mail or the World Wide Web using the MIME content type 'application/x-sdp'. For invitations, where a response is required, SDP descriptions may be sent using the session initiation protocol (SIP) [33].

The **real-time streaming protocol** (RTSP) [34] is an application protocol under development within the IETF for control of delivery from streaming servers such as audio or video servers. Sources of data can include both live data feeds and stored clips. The protocol supports:

- retrieval of media description from the media server — the session-description format may take several formats including SDP,
- a means of choosing delivery channels (e.g. UDP, multicast UDP or TCP) and delivery mechanisms based on RTP,
- invitation of a media server into a session,
- introduction of new media streams from a server already participating in a session,
- start, stop, pause of media delivery.

4.2 Data handling

The data-handling object manages discrete interactions between peer applications according to the QoS requirements specified by the application object.

An object of special significance that makes use of the data-handling object is the session-control object.

Examples of applications that have implemented some of the functionality in the data-handling object include wb (the white-board), nt (the shared text editor [35]), and work on distributed simulations in DIS. Examples of protocols developed/proposed include SRM, a scalable reliable multicast [36], the distributed interactive simulation (DIS) protocol [37], and a suggestion that a virtual reality transport protocol (VRTP) is required [38].

Services provided by the data-handling object include QoS-driven delivery of discrete interactions and co-ordinated access to shared resources.

4.2.1 QoS-driven delivery of discrete messages

This service manages delivery according to requested reliability and timeliness requirements. The data-handling object is able to dynamically enhance (or replace) the underlying unreliable service with additional reliability where necessary. This may, for example, require QoS-driven protocol selection where the application protocol is determined by the requested QoS and the current environmental conditions. A variety of techniques can be used to make interactions reliable.

- Resilience against packet loss

  — Re-transmissions can guarantee delivery or notification of failure. TCP is suitable for point-to-point delivery, but for delivery to groups other techniques supporting a number of reliability modes are required (none, at least one, *n* out of *m*, all). 'All' is only appropriate if the members of the group are reliably known:

  — redundancy can be used to distribute information across several packets (normally used for isochronous interactions, e.g. RAT which is discussed later, or to repeat information at regular intervals, e.g. nt),

  — reservations are suitable for isochronous interactions or a series of discrete interactions.

- Reordering of packets

  Packets may arrive out of sequence and require reordering. Indeed different receivers may experience packets arriving in different orders. Reordering of packets from a single source requires buffering (which introduces delay) and RTP time-stamps (the same technique can be used to address jitter — see section 4.3).

  This technique is suitable for handling mis-ordering arising from bounded delays. For unbounded delays re-transmissions must be used.

  Ordering of packets from multiple sources requires time-stamps and clock synchronisation (see section 4.6). Potentially large delays are introduced as receivers are forced to wait for the slowest source.

- Fairness

  Fairness is often used to refer to equitable sharing of resources. However, in the context of this paper fairness refers to the timing of delivery of information. Consider for example a trading system where prices change rapidly with the market. The delivery of any

offer must be timed such that all parties have the same amount of time to react. This can be achieved in the following way:

— by buffering received packets so that the time between transmission and presentation is the same (within some error bound), irrespective of distance or network conditions — this requires global clock synchronisation, and security measures to prevent information being disclosed early,

— alternatively, delivery of an offer-to-sell could be time-stamped at the moment of presentation and any offer-to-buy time-stamped at the moment of response, the decision system then waiting long enough before analysing responses to be sure that all users have had a chance to respond. The decision system then places greater priority on those users who responded fastest. The fastest responses would be those that were turned around at the end-system most quickly and not necessarily those to arrive at the decision system first. This approach does not require global clock synchronisation but it does require a secure time-stamping system.

Neither of these approaches guarantee delivery. They merely ensure that delivery is fair for those packets that arrive within some conservative time bound. However, they can make use of IP multicast and they are limited only by a single round-trip time to the slowest site.

Fair guaranteed delivery requires a two-phase commit protocol for each transmission resulting in a total transmission time for the request and the response of at least four round trips to the slowest site.

4.2.2 Co-ordinated access to shared resources

This service manages the right to manipulate a particular object. That object may be a specific application object such as a white-board or a shared editor, it may be a logical object such as the conference floor, or indeed the session itself, or it may be a virtual object in a shared virtual world.

The challenge is to maximise the perception of consistency for users. Possible approaches include:

- locks enabling control to pass from one user to another, which can be provided for use on a first-come-first-served basis or they can be provided to a controller, to allow other users to request rights and allow the controller to give, deny or withdraw rights and delegate rights,
- regular redundant updates, e.g. the text editor, nt, regularly sends the whole of the current line so that inconsistency can be quickly ironed out,
- leave it to the users, e.g. the simplest floor control is to allow conference users to organise themselves by voice communication.

Additionally some technique is required to discover inconsistency and re-establish consistency.

4.2.3 Synchronisation

In a number of cases it is important to be able to synchronise discrete events with the presentation of a particular video or audio frame. If this is required then a play-out buffer for discrete events would be required. Techniques for inter-media synchronisation are discussed in section 4.5.

4.3 Audio

The audio-handling object manages the movement of audio between the audio device and the network. It interacts with the audio device through an API to the two device-driver buffers. Audio samples are passed in blocks from the input buffer for processing or to the output buffer for playback. The minimum block size is determined by the workstation's processing power — the smaller the block size the more frequently the application has to read and write blocks. Blocks of samples taken from the device-driver buffer must be encoded and packetised before they can be transmitted. Conversely packets received from the network must be de-packetised, and decoded before being passed in blocks to the device-driver buffer.

This simple model may be enhanced by additional functions including play-out adaptation, mixing, silence suppression and echo cancellation, as shown in Fig 4.

4.3.1 Packetisation

Packetisation is a simple process of placing audio frames into RTP packets. The key question is how big the packet should be. The bigger the packet, the longer the delay for sampling and encoding, packetisation and transmission. The smaller the packet, the larger the relative overhead of the packet header and therefore the poorer the bandwidth utilisation. The IETF is working on header compression [39].

4.3.2 Play-out adaptation and reordering

A play-out buffer at the receiver is used to smooth out any jitter and if necessary reorder frames arriving out of sequence. Selection of the best playback point (or effective buffer size) depends on striking the right balance between minimising end-to-end delay (by setting an early playback point) and minimising the discard of late frames (by setting a later playback point). The right balance will depend on

whether the application is conversational (requiring low delay) or a simple one-way stream.

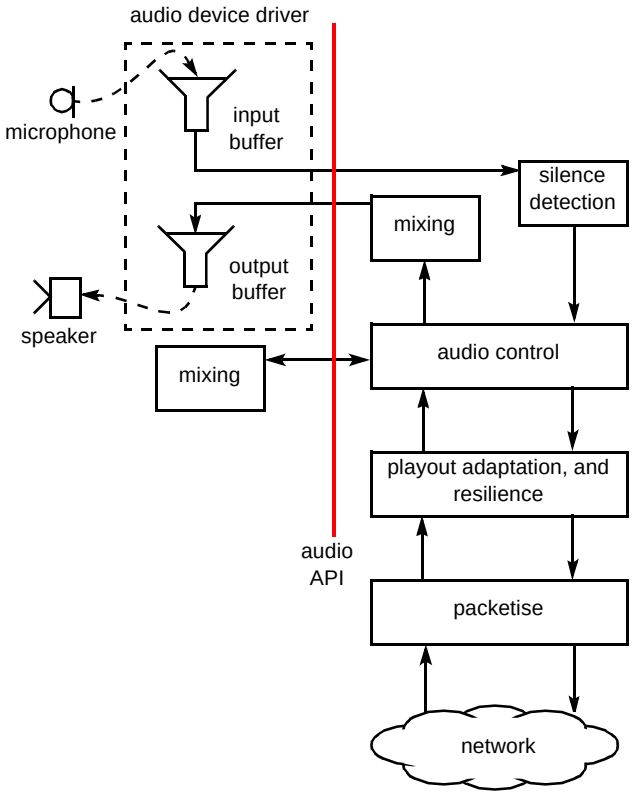

Fig 4 Audio handling.

The best results are obtained if the play-out point dynamically adapts to the current mean variance in delay. The receiver can estimate the inter-packet arrival time variance using exactly the same technique as TCP uses to estimate the round trip time [40].

The network is not the only source of jitter; under a time sharing operating system, CPU starvation can quite easily lead to the play-out buffer becoming empty, resulting in gaps in the audio, but by monitoring a history of scheduling anomalies experienced by the audio-handling object it is possible to adapt the play-out buffer accordingly [41].

4.3.3 Coding

Coding aims to represent the media signal as well as possible in as few bits as possible. Also since re-transmissions are generally inappropriate for streamed media, the coding should aim to be robust to transmission errors. The main compression techniques can be classified as:

- wave-form coding, which attempts to preserve the audio signal, the main examples of wave-form coding being the PCM family of codecs,
- knowledge-based coding, which attempts to model and synthesise the source of the sound, e.g. the voice or possibly an instrument.

PCM exploits the amplitude distribution characteristics. Since there is a high probability of zero and low amplitudes, these quantisation levels are coded more accurately than for higher amplitudes.

Adaptive differential PCM (ADPCM) exploits the slow variation in amplitude levels by coding the difference between the actual value of the current sample and a prediction of the sample based on a knowledge of earlier samples. It does this in a number of ways. Firstly, since the correlation between adjacent samples is especially high, only the difference in value needs to be sent. This is further improved through the use of a predictor based on a number of earlier samples to predict the next value. In this case only a correction against the prediction needs to be transmitted. The predictor can also be made to adapt to the frequency content because voiced and unvoiced phonemes have different short-term frequency characteristics. Secondly, at any given time only a limited subset of the total dynamic range (range of amplitudes) is assigned quantisation levels. Using adaptive quantisation, the magnitude of the correction factor can be changed over time dependent on the dynamics of the signal.

Knowledge-based coding has mainly focused on modelling speech. It works by generating a synthesis filter to model the vocal tract. In the case of linear predictive coding (LPC) an excitation source is passed through the synthesis filter. The synthesis filter is defined in terms of its weights or coefficients of a finite impulse response filter. These filter parameters are calculated by the encoder and transmitted to the receiver. Samples are processed in blocks, or frames typically of the order of 20 — 30 ms long. New coefficients are sent to the receiver to reflect the time-varying properties of speech. In addition to these coefficients, other parameters, such as the current gain (volume) and pitch, may also be sent. Simple linear predictive coding uses a very basic excitation signal. Voiced sounds (vowels and vowel-like sounds) are periodic, so a periodic train of pulses is used as the excitation source. Unvoiced sounds are aperiodic and so in this case white noise is used as the excitation source. The result is a reptilian voice quality referred to as synthetic quality.

The family of residual excited linear prediction codecs (which include GSM) generates a so-called residual to be used as the excitation signal. The residual is generated by subtracting the previous filter output from the speech input. Because most of the information in the residual is below 1 kHz this can be coded and transmitted efficiently. Of the 13 kbit/s approximately 5 kbit/s is used for the LPC frequency spectrum analysis with the remainder used to send the excitation signal. Code excited linear prediction

(CELP) makes a further improvement by encoding the excitation signal using codewords from a shared code-book.

Table 1 summarises some of the main coding standards [42]. The mean opinion scores (MOS) are merely indicative because they were obtained from different subjective tests using different test material and were evaluated under optimum conditions. The complexity is measured relative to the complexity of G.711. The delay includes both the frame size and the look-ahead used in the coding algorithm.

Table 1 Speech coding standards.

Standard	Coding type	Bit rate kbit/s	MOS	Complexity	Delay (ms)
G.711	PCM	64	4.3	1	0.125
G.726	ADPCM	32	4.0	10	0.125
G.728	LD-CELP	16	4.0	50	0.625
GSM	RPE-LTP	13	3.7	5	20
G.729 G.729A	CSA-CELP	8	4.0	30 15[4]	15
G.723.1	A-CELP MP-MLQ	6.3 5.3	3.8	25	37.5
US DoD FS1015	LPC-10	2.4	synthetic	10	22.5

Both PCM and LPC-based compression techniques are aimed at speech. As a wave-form coding PCM can, however, give adequate music quality. As a family of codecs that models speech, LPC is generally unsuited for compressing music or other kinds of audio [5].

For good quality non-speech audio, the leading standard is MPEG audio layer 3, which is a transform coding providing almost CD quality at 64 kbit/s per monophonic channel. It can operate at bit rates down to 8 kbit/s.

4.3.4 Resilience

The quality of transmitted speech is critical to successful person-to-person communication. Loss of more than 20 ms of audio is audible so typically any packet loss will be audible to the listener [7], therefore any measures to minimise the impact of packet loss are desirable. These would include receiver-only reconstruction mechanisms such as white-noise substitution or LPC reconstruction as in the robust audio tool known as RAT [43].

Experiments with RAT showed that a receiver-only voice reconstruction mechanism is suitable for low loss rates (20% and below) and small packet sizes (of around 20 ms). Listeners, however, preferred LPC reconstruction for higher loss rates (up to 40%) or for packet sizes of 40 and 80 ms. RAT works by placing the current audio frame and LPC component of the next audio frame in the current packet. If a packet is lost then the LPC component from the previous packet can be substituted. If subsequent packets are also lost then the LPC component could be repeated but only for as long as the speech characteristics remain unchanged — typically 80 ms, the average length of a phoneme [44].

Use of LPC redundancy is particularly effective because the LPC component includes 60% of the information but uses a fraction of the available bandwidth [44].

4.3.5 Mixing

The receiver audio-handling object can mix multiple audio streams by correlating time-stamps (see section 4.5) and then combining decoded samples within blocks before passing to the play-out buffer.

4.3.6 Silence suppression

Silence suppression can be used in the transmitter to reduce data rates during periods of silence. This is particularly important if a workstation is receiving multiple streams (e.g. using IP multicast) and has limited bandwidth and processing power.

4.3.7 Echo cancellation

Conversation is very difficult if a user can hear their own voice more than a few tens of milliseconds after they speak [45]. This problem may arise if your voice traverses the network, is heard through the speakers and picked up by the microphone at the far end and then returns. Simple solutions to this problem include use of a headset, careful positioning of microphone and speakers, or use of push-to-talk to enforce half-duplex communication. A more ambitious approach is to place an echo cancellor between the audio-out and audio-in to sense the delay in the room between the output signal on the speakers and the input signal on the microphone. The cancellor then attempts to remove the echo by adding the same signal but with its phase reversed and with the detected delay to the input. Unfortunately echo cancellation is very difficult because the output signal is transformed by the room and so when it arrives at the microphone it may not be recognised as the original output signal.

[4] This figure is for G.729A, a low-complexity variation of G.729.

[5] G.728 is surprisingly good at coding music.

4.4 Video

The video handling object manages the movement of video frames between the network and the video driver. The main functions are shown in Fig 5 and discussed below.

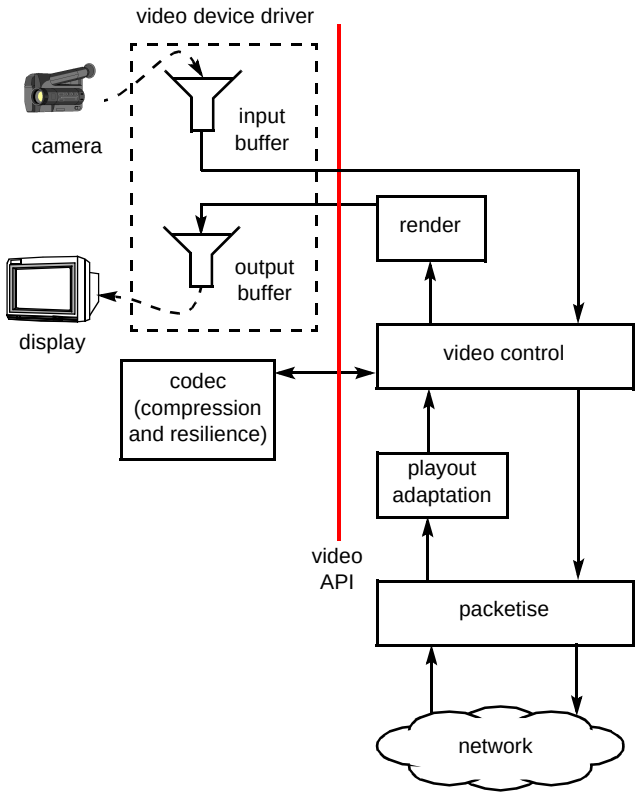

Fig 5 Video handling.

4.4.1 Packetisation

As with audio the size of the RTP packet is important, with there being a trade-off between delay and overhead. However, it is also important for error resilience to align the packet boundaries with self-contained parts of the video bit-stream as far as possible.

A video frame is organised as a set of groups of blocks (GOB). Each GOB holds one or more macro blocks (MB). GOBs are independent of their neighbours apart from the effect of motion compensation which crosses GOB boundaries. The result of this hierarchical structure is that information in the frame header is needed to decode GOBs and information in the GOB header is needed to decode a macro-block. Unfortunately a frame or even a GOB may be too large to fit in a single packet so the macro-block is taken as the unit of fragmentation, i.e. RTP packets must contain a whole number of macro-blocks. To allow independent processing of each packet some state information for the frame header and the GOB header is carried with each packet.

4.4.2 Play-out adaptation

As for audio a play-out buffer can be used to minimise the effects of jitter and to reorder packets. The timing of RTP packets is known from the RTP time-stamp which encodes the sampling instant of the first video image contained in the RTP packet. In the case of H.261 the RTP time-stamp is based on a 90 KHz clock rate, which is a multiple of the 29.97 Hz natural frame rate used for H.261 [46]. If a video image occupies more than one packet, the time-stamp will be the same on all those packets and serves to identify the frame to which the data belongs. If a packet contains more than one frame then a temporal reference field in the header of each frame of video, is used to indicate the correct time to display that frame relative to the previous one. Essentially this field indicates how many frames have been dropped since the last one, and a decoder should in principle wait the appropriate number of frame intervals before passing the new decoded frame for rendering on to the display device. This will address any jitter introduced during coding and a limited amount of network jitter. However, to fully address network jitter, packets should be buffered (and reordered) using the RTP time-stamp before decoding.

4.4.3 Rendering

Rendering is the preparation of an image for display on the chosen device. This includes doing any necessary work to handle any characteristics of the display device that do not match those of the image (e.g. dithering a 24-bit image on to an 8-bit palette).

4.4.4 Compression

The amount of data required to transmit images is usually orders of magnitude greater than that required for text, and there is a clear need for image compression. This is particularly important for moving images where a new displayed image is required ten or more times a second.

The two main ways of compressing image data are removal of statistical and perceptual redundancy.

In the first, the fact that picture elements or ‘pixels’ which are close in space and/or time tend to have similar values is exploited. A key technique here is predictive coding where pixels (or more generally, data) already received are used to predict the next to arrive. Since the decoder and encoder both have access to the same previous data and can construct the same prediction, the encoder only has to send the prediction error, which usually has a much lower variance than the original data and requires fewer bits to transmit. Even more compression can be achieved by exploiting the visual perception characteristics of the human eye/brain system. For example, we cannot resolve the colour of objects to the same resolution as their brightness,

small errors are masked by the close proximity of detail in an image, and we can tolerate more distortion of high spatial frequencies (high detail) than low ones (the broader sweep of light and dark.

The last characteristic is exploited particularly well by transform coding, and wavelet-based techniques, both of which map the image from the time domain represented by a series of samples in space to spatial frequency domains where appropriate quantisation can be applied which matches perceptual criteria.

Standardised coding algorithms are almost exclusively based on a discrete cosine transform (DCT) operating on blocks of 8 × 8 pixels, including JPEG [47], H.261 [48], H.263 [49], MPEG1 and MPEG2 [50].

JPEG is designed for single images, although it can be used for moving images simply by encoding them as a sequence of essentially separate stills. It employs only 'intra-frame' coding techniques. Algorithms for moving images also use 'inter-frame' techniques where the last decoded image is used to predict the next one. The simplest inter-frame technique is conditional replenishment where a pixel or block of pixels is only updated if it has changed significantly since the last decoded image. In a typical videophone scene only 20 to 30% of the blocks might need updating in any frame. A more advanced inter-frame technique first standardised in H.261 is motion compensated temporal prediction, where blocks of pixels from the last decoded frame are moved around to track the motion of objects in the scene and construct a better prediction. Subsequently the prediction error is compressed using similar transform coding methods to JPEG.

More complex temporal prediction schemes were introduced in MPEG1 and MPEG2, which were intended for stored or broadcast video at higher data rates, and use bi-directional temporal prediction (B-frames). This technique increases compression, but at the expense of extra coding delay as some frames have to be stored and processed out of their normal temporal order. Thus B-frames are not normally used for real-time conversational services where low delay is crucial. H.263 was initially developed for use in PSTN videophones, and adds other refinements such that H.263 is perhaps twice as efficient as H.261. Although MPEG1/2 were developed primarily for stored and broadcast purposes at higher bit rates (typically above 1 Mbit/s), and H.261/3 for conversational use at low bit rates (typically 20 — 384 kbit/s), with suitable choice of image resolution and coding parameters their application domains and bit-rate ranges can be extended to overlap.

4.4.5 Resilience

The downside of compression is increased sensitivity to errors or partial loss of data, such as through packet loss. One reason is the use of variable length coding where code-words of varying lengths are used according to the probability of use of that code-word. Any error in the received bit sequence may cause a code-word of a different length to be decoded which then causes a knock-on effect of misinterpreting and wrongly decoding all the following bits. In a still image compression scheme such as JPEG this can scramble the rest of the picture after the error in the worst case, though sometimes the code-words get back in step and the result is a distortion of only a few blocks, and/or side-ways displacement of following blocks. For a moving JPEG sequence there is no effect of following pictures as the system resynchronises at the start of each new picture. However, for algorithms using temporal prediction the situation is much more serious as an error in one picture then propagates into the next one through the prediction process, and the entire remaining sequence can be destroyed.

Techniques using conditional replenishment give better error resilience but less compression. To stop errors propagating indefinitely some blocks or even complete pictures in a sequence can be intra-coded instead of inter-coded; intra-coded blocks or images are reconstructed without reference to any previous and possibly wrong data. MPEG1 and MPEG2 typically include a so-called I-frame (an intra-coded frame) every 20 or so frames, which ensures recovery from errors within a second at normal frame rates. However, I-frames are several times more expensive in terms of bits than other frames, so there is a trade-off between overall compression efficiency and error resilience. To help with this, the video RTCP profiles have provision to signal the loss of individual packets, or request intra-coding of an entire frame. However, the usefulness of these features in a large multicast session is a matter for debate.

4.4.6 Layered video

Often multiple parties need to receive video at a range of different bit rates depending upon the (time-varying) network capacity to their node. To address this, the idea of an 'embedded' or 'scalable' video bit-stream has developed, where a compressed video stream is coded as a hierarchy of separately identifiable and decodable sub-bit-streams. If each incremental stream is transmitted on a separate multicast address for example, a receiving node can join as many streams as is possible before it overloads the network capacity available to it.

The principal modes of scalability are temporal, spatial, and signal/noise ratio (SNR). Thus for temporal scalability the lower bit-rate streams might only provide a frame rate of 7.5 frames/s, and higher layers 15 or 30 frames/s. Spatial scalability provides a range of basic image resolutions. SNR scalability implies the use of coarser quantisation (of DCT coefficients, for example) for lower layers. The process of building the higher layers from the lower is effectively

another form of prediction process, and similar problems of sensitivity to errors arise. Again, the more efficient the prediction and hence compression becomes, the lower the tolerance to errors. It is still a matter for debate as to whether overall system performance in a lossy network is optimised by going for a high compression scheme such as H.263 with an appropriate level of error protection and re-synchronising redundancy, or a lower compression but more inherently robust algorithm. An example of the latter, which also lends itself well to scalability, is that of Chaddha et al [51], which uses wavelets for spatial scalability combined with conditional replenishment for temporal compression and scalability, and a hierarchical vector quantisation scheme for SNR scalable intra-frame compression.

4.5 Synchronisation

There are two main forms of synchronisation. Intra-stream synchronisation is the association of a particular bit within the stream with a particular time-slot. RTP time-stamping provides a suitable basis — the declared payload type indicates the timing between bits within a packet; the media time-stamps indicate timing between packets. Inter-stream synchronisation is about synchronising different streams from potentially different sources.

Lip synchronisation is an example where the streams are of different media but where the sources are (usually) the same. To achieve adequate synchronisation a method is required of synchronising the play-out points in the audio and video play-out buffers. This can be done, as shown in Fig 6, using inter-media communication to transfer the desired play-out delays between the audio and video-handling components [52].

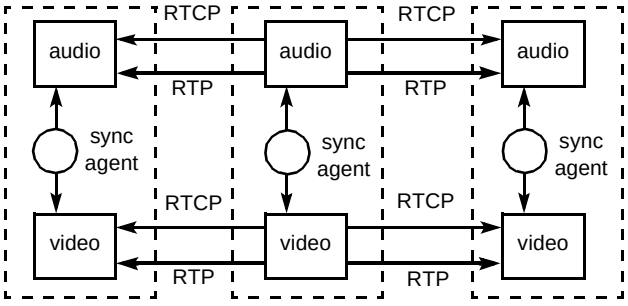

Fig 6 Media agent synchronisation.

RTP is used for transmission of audio and video packets. The time stamp is media specific and different reference clocks may be used for audio and video, so RTCP control messages provide individual mapping between the media and the network time protocol (NTP) time stamps [53].

Because NTP is a clock synchronisation protocol, this solution is suitable for streams from different sources. (To achieve inter-media synchronisation for streams from a single source, it would be sufficient to map time-stamps on to a common local clock.)

4.6 Application support tools

A number of tools are useful to support both conversational and streamed real-time services. Three important tools are a personal directory, a session directory and a diary.

4.6.1 Personal directory

Whether for asynchronous (message-based) communication or for synchronous (real-time) communication, the user needs ready access to other users' contact details such as audio, video, e-mail, Web page, personal virtual world, etc. A personal directory providing a local cache, but capable of federating with other directories, is required.

Ideally each person would have one (or more [6]) personal identifier(s), which is (are) independent of the particular application being used to contact them. It should be left to the peer session-control objects to negotiate the best application for communication (based on the application selected by the caller and the applications supported by the called party). The IETF standard for directories is the lightweight directory access protocol (LDAP) [54] which is a simplification of X.500 [55].

4.6.2 Session directory

A session directory enables users to find out what events are currently taking place or will in the future take place. These 'events' might include meetings, conferences, radio/TV programs, exhibitions of virtual reality art, or virtual world spaces that can be explored together. Like a networked games competition, the event might be one off, or like an information delivery channel it might be continuously available. The event might be private to a few selected people or it may be public. In any case, the tool should be intelligent and personalisable in what it selects for presentation to the user and the way it presents that information.

SDR must be the best-known tool that begins to address these requirements, though Multikit [56] is more advanced.

[6] Additional identifiers may be useful for a number of reasons, including user privacy or prioritised call handling.

4.6.3 Diary

Having seen a future event of interest, you could copy this to your diary. When the appropriate time arrives you could join the event by clicking on the entry in your diary — or indeed your diary could initiate joining the event for you. After the event the entry in your diary would provide a pointer to any recorded information, e.g. textual minutes or audio, video or virtual-world recordings. The IETF is developing standards to allow diary applications to inter-operate with each other and with other applications [57].

Together a personal directory, a session directory and a diary would provide a powerful collection of support tools for networked conversational and streamed services.

5. Security

There are two approaches to network security — prevent unauthorised access to the network or encrypt messages sent over the network.

Within an intranet, administrative and time-to-live (TTL) scoping [58] may be used to prevent multicast streams being transmitted beyond the intranet. But since IP multicast allows open and unknown receiver groups, the only way of securing multicast communication on the wider Internet is encryption. Generally, this means using fast encryption using shared session keys on the media streams and public-key cryptography to distribute the session keys [59].

To securely distribute the session keys only to those who have the right to receive them, each party must be authorised. Depending on the type of service, this may happen by authentication or by payment. Either way, once authorisation has taken place, a shared secret key is created and made known (by public-key cryptography) only to the new party and to the session owner. This secret shared key is then used by the session owner to send the session keys to the new party. The session key is shared by all parties to the session. The main problems with this are twofold.

- Lack of scalability — for each new party, the session owner needs to:

  — authorise (by authentication or payment),

  — create a shared secret key and make known using public-key cryptography,

  — encrypt the session keys using the secret shared key.

  RFC1949 ‘scalable multicast key distribution’ [60] proposes a scalable method for distributing keys based on the core-based tree (CBT) multicast protocol [61]. It requires CBT only for key distribution and not for transmission of media streams. The proposal requires slight changes to the CBT protocol and to the IGMP protocol.

- Authentication of arbitrary users requires a global certification hierarchy capable of certifying all potential users. The X.509 [62] certification standard provides a technical basis; however, this remains an enormous task and requires some means of unambiguously identifying a person before they can be issued with a certified public key (that is a public key signed by the certification authority).

For some services, pay-as-you-go charging methods such as micro-payments may offer an alternative to authentication as a basis of authorisation.

6. Conclusions

The IETF’s integrated services architecture and a loosely coupled approach to session control provide a basis for supporting a wide range of real-time services. The difficult issues concern quality of service guarantees, performance and scalability.

While ISA promises resource reservations there remains some debate as to whether the ability to support network QoS guarantees is worth the consequent loss of performance that necessarily arises from complex scheduling mechanisms. Some argue that guarantees only make sense if there is an associated charge and so any inefficiency arising can be covered in the charge. The argument then becomes a market decision. QoS guarantees for operating system resources are still a matter of research. With the rapid growth of the Internet, performance and scalability (particularly of discrete interactions for session control and data control) are set to become increasingly important. The problem of supporting large-scale shared distributed virtual worlds encapsulates many of the issues. A significant forum for the discussion of these issues is the recently formed Large Scale Multicast Applications (LSMA) working group in the IETF [63].

Acknowledgements

The authors would particularly like to thank colleagues at BT Laboratories and at University College London, for their valuable comments.

References

1 Babbage R, Moffat I, O’Neill A and Sivaraj S: ‘Internet phone — changing the telephony paradigm’, BT Technol J, 15, No 2, pp 145—157(April 1997).

2 Saltzer J et al: ‘End-to-end argument in systems design’, ACM Transactions on Computer Systems (November 1994).

3 Apteker R T et al: ‘Video Acceptability and frame rate’, University of the Witwatersrand, IEEE MultiMedia (Fall 1995).

4 ITU-IT Recommendation G.114: 'Mean one way propagation time', Melbourne (1988).

5 IEEE Standard P1278.2: 'Standards for distributed interactive simulation — communication architecture requirements', (1995)

6 Jardetzky P W, Sreenan C J and Needham R M: 'Storage and synchronisation for distributed continuous media', Multimedia Systems, 3 , No 3 pp 151—161 (September 1995).

7 ETSI Rec ETS 300 580-3.8: 'European digital cellular telecommunications system: Substitution and muting of lost frames for full rate speech channels (GSM 06.11)', (1994).

8 Braden R, Clard D and Shenker S: 'Integrated services in the Internet architecutre: an overview', IETF RFC 1663 (July 1994).

9 O'Neill A: 'Internetwork futures', BT Technol J, 15, No 2, pp 226—240 (April 1997).

10 Braden R et al: 'Resource reservation protocol (RSVP) — version 1 functional specification', IETF, RSVP working group work in progress (draft-ietf-rsvp-spec-14) (November 1996).

11 Deering S: 'Host extensions for IP multicasting', IETF RFC 1112 (August 1989).

12 Schulzrinne H, Casner S, Frederick R and Jacobson V: 'A transport protocol for real-time applications', Request for comments RFC 1889, IETF (January 1996).

13 Clark D D and Tennenhouse DL: 'Architectural considerations for a new generation of protocols', Proc ACM SIGCOMM, Philadelphia, Pennsylvania (1990).

14 Quinn B and Shute D: 'Windows sockets network programming', Addison-Wesley, Reading, MA (1996).

15 'Windows Sockets 2 API: the application programming interface specification', Filename:WSAPI22, current revision 2.2.0 (10 May 1996).

16 'Windows Sockets 2 Protocol Specific Annex', Filename: WSANX203, current revision 2.0.3 (10 May 1996).

17 Schulzrinne H: 'Operating system issues for continuous media', ACM Multimedia Systems (October 1996).

18 Mercer C W: 'An introduction to real time operating systems: scheduling theory', Carnegie Mellon University, Pittsburgh (1996).

19 'Real time mach', Carnegie Mellon University, http://www.cs.cmu.edu/afs/cs/project/art-6/www/rtmach.html

20 'SUMO support for multimedia in operating systems', Lancaster University, http://www.comp.lancs.ac.uk/computing/research/sumo/

21 McCanne S and Jacobson V: 'vic: a flexible framework for packet video', in Proc of ACM Multimedia '95 (November 1995).

22 Casner S and Deering S: 'First IETF Internet audiocast', Computer Communications Review, 22 (July 1992).

23 The whiteboard wb, ftp://ftp.ee.lbl.gov/conferencing/wb

24 Handley M: 'The session directory tool SDR', http://mice.ed.ac.uk/archive/sdr/html

25 McCanne S, Jacobsen V and Vetterli M: 'Receiver driven layered multicast', University California, Berkeley and Lawrence Berkeley National Laboratory, ACM SIGCOMM, Stanford, Cal (August 1996).

26 Handley M, Wakeman C and Crowcroft J: 'The conference control channel protocol', ACM SIGCOMM '95, Cambridge, Mass (September 1995).

27 Benford S, Greenhalgh C and Lloyd D: 'Crowded collaborative virtual environments', to be presented at ACM CHI'97.

28 Crowcroft J, Hardman V and Lewis D: 'Pricing Internet services', submitted to IEEE Magazine.

29 ITU-T draft recommendation H.323: 'Visual telephone systems and equipment for local area networks which provide a non-guaranteed quality of service', (May 1996).

30 ITU-T draft recommendation T.120: 'Data protocols for multimedia conferencing', (February 1996).

31 Handley M and Jacobson V: 'SDP — session description protocol', IETF MMUSIC Working Group Internet Draft, draft-ietf-mmusic-sdp-02.txt (21 November 1996).

32 Handley M: 'SAP session announcement protocol', IETF MMUSIC Working Group Internet Draft: draft-ietf-mmusic-sap-00.txt (19 November 1996).

33 Handley M, Schulzrinne H and Schooler E: 'SIP: session initiation protocol', IETF MMUSIC Working Group Internet Draft: draft-ietf-mmusic-sip-01.txt (2 December 1996).

34 Schulzrinne H: 'A real-time stream control protocol (RTSP)', IETF Internet Draft: draft-ietf-mmusic-stream-00.txt (26 November 1996).

35 Handley M: 'Network text (nt) — a scalable shared text editor for the Mbone', Department of Computer Science, University College London (1995).

36 Floyd S, Jacobsen V, McCanne S, Liu C-G and Zhang L: 'A reliable multicast framework for light-weight sessions and application level framing', Proc ACM SIGCOMM '95, Cambridge, Mass (1995).

37 IEEE Standard 1278: 'IEEE standard for information technology — protocols for distributed interactive simulation applications', (1993).

38 Brutzman D P, Macedonia M R and Zyda M J: 'Internetwork infrastructure requirements for virtual environments', Presented at the Virtual Reality Modelling Language Symposium, San Diego Supercomputer Center (SDSC), San Diego, California (December 1995).

39 Casner S and Jacobson V: 'Compressing IP/UDP/RTP headers for low-speed serial links', IETF Internet, Draft: draft-ietf-avt-crtp-01.txt (25 November 1996).

40 Stevens W R: 'TCP/IP illustrated, volume 1 — the protocols', Addison-Wesley (1994).

41 Kouvelas I and Hardman V; 'Overview workstation scheduling problems in a real time audio tool', Usenix Annual Technical Conference, Anaheim, California (January 1997).

42 Wong W T K, Mack R M, Cheetham B M G and Sun X Q: 'Low rate speech coding for telecommunications', BT Technol J, 14, No 1, pp 28—44 (January 1996).

43 Hardman V, Kouvelas I, Sasse M A and Watson A: 'Robust audio over the Internet — analysis and implementation', Research Note RN/96/8, Dept of Computer Science, University College, London (February 1996).

44 Hardman V, Sasse M A, Handley M and Watson A: 'Reliable audio for use over the Internet', Proceedings of INET 95, Ohahu, Hawaii (1995).

45 ITU-T Rec G.114: 'One-way transmission time', (1996).

46 Turletti T and Huitema C: 'RPT payload format for H.261 video streams', IETF RFC 2032 (October 1996).

47 Penbaker W B and Mitchell J L: 'JPEG still image data compression standard', Van Nostrand Reinhold (1992).

48 ITU-T Rec H.261: 'Video codec for audiovisual services at p × 64 kbit/s', (1993).

49 ITU-T Rec H.263: 'Video coding for narrow telecommunications channel', (1993).

50 International Standard IS 11172-2: 'Coding of moving video and associated audio at rates up to about 1.5 Mbit/s part 2: video', (1995)

51 Chadda N, Chou P and Meng T: 'Scalable compression based on tree structured vector quantisation of perceptually weighted generic block, lapped and wavelet transforms', Proc IEEE International Conference on Image Processing (October 1995).

52 Kouvelas I, Hardman V and Watson A: 'Lip synchronisation for use over the Internet — analysis and implementation', presented at Globecom 96 (1996).

53 Mills D L: 'Network time protocol (version 3) specification, implementation and analysis', IETF RFC 1305, Network Working Group (March 1992).

54 Yeong W, Howes T and Kille S: 'X.500 lightweight directory access protocol (version 2)', IETF RFC 1777 (March 1995).

55 ITU-T Rec X.500: 'The directory: Overview of Concepts, Models and Service', (1988).

56 Multikit, http://www.lvn.com/multikit

57 Dawson F: 'MIME calendaring and scheduling content type', IETF Network Working Group Internet Draft: draft-ietf-calsch-csct-00.txt (26 November 1996).

58 Meyer D: 'Administratively scoped IP multicast', IETF MBONE Deployment WG Internet Draft: draft-ietf-mboned-admin-ip-space-0.txt (December 1996).

59 Phoenix S J D: 'Crytography, trusted third parties and escrow', BT Technol J, 15, No 2, pp 45—62 (April 1996).

60 Ballardie A: 'Scalable multicast key distribution', IETF RFC 1949 (1996).

61 Ballardie A, Reeve S and Jain N: 'Core based trees (CBT) multicast— protocol specification', Internet Draft (September 1996).

62 ITU Rec X.509: 'The directory — authentication framework', Geneva (November 1993).

63 'IETF large scale multicast applications (LSMA) Charter', http:// www.ietf.org/html.charters/lsma-charter.html

Steve Rudkin graduated from the University of Cambridge in 1985 with a BA in electrical and natural sciences. After completing an MSc in computation at the University of Oxford in 1986 he joined the SEMA group to work on real-time embedded software.

He moved to BT Laboratories in 1987 to research the use of formal and object-oriented methods in distributed systems. Since 1993 Steve has led the distributed systems group and its research into Web-based commerce, and multimedia on the Internet.

Andrew Grace joined BT Laboratories in 1988 having graduated from Plymouth Polytechnic with a BEng in communication engineering. He spent the first six years of his career applying novel data mining and data visualisation techniques to large quantities of fault and performance data from BT's core transmission network.

In 1994 he moved into the distributed systems group to lead a research activity looking at end-to-end quality of service issues within open distributed systems.

He is currently interested in real-time Internet applications, particularly video enhanced World Wide Web. He is a Chartered Engineer, and a Member of the IEE.

Mike Whybray joined BT Laboratories in 1977, working on reliability physics, and in 1983 moved to the visual telecommunications division where he led a group whose activities included developing videophones for deaf people, writing software for DSP based videophones, and building demonstrators of intelligent surveillance systems and synthetic 'talking head' displays.

He currently leads a group within advanced applications and technologies developing new speech and image coding algorithms and standards.